

Recursive Language Models

Alex L. Zhang, Tim Kraska, Omar Khattab

Massachusetts Institute of Technology (MIT)
Department of Electrical Engineering and Computer Science

April 22, 2026



Start of 2026: Context management in language models

Authors
Sebastian X

January 1, 2026

Recursive Language Models: *the* paradigm of 2026

How we plan to manage extremely long contexts

LLM agents have become significantly more useful over the course of this year. They are now capable of implementing complex changes in large codebases autonomously, often reading and editing dozens of files, searching the web, and maintaining context even over the course of multiple such complex requests.

These capabilities require the use of vast numbers of tokens.

But that, in turn, is difficult for current LLMs: per-token costs rise linearly with the context length, while the performance of even the best models drops with it. A well-known phenomenon at this point is context rot, the reduction of LLM capabilities as contexts grow in size. And even though changes to architecture and training data have caused, and will continue to cause, much progress to address these challenges, there is one thing that is complementary to both, and has consistently been a huge multiplier to LLMs' effective context length: scaffolding.

Jan 1. Prime Intellect reveals they are working on RLMS.

Deedy @deedydas · Jan 6

Claude Code is one of the worst named products. It's not just for **code**.

Watch this Hermès ad.

30 seconds. 8 shots. Voice. Music. Even text branding burned in with ffmpeg. Clopus 4.5 wrote a script, orchestrated ElevenLabs, Google Veo 3, downloaded music and made this from [Show more](#)



0:23

281 331 4.8K 1M

Past few months: people are trying Claude Code for non-coding tasks.

Cursor @cursor.ai

Cursor's agent now uses dynamic context for all models.

It's more intelligent about how context is filled while maintaining the same quality. This reduces total tokens by 46.9% when using multiple MCP servers.

Dynamic context discovery of MCP tools reduced total tokens by 46.9%

● System Instructions ● Tools ● MCP File access



Before

After

46.9% fewer total agent tokens

3:57 PM · Jan 6, 2026 · 667K Views

137 354 2.9K 821

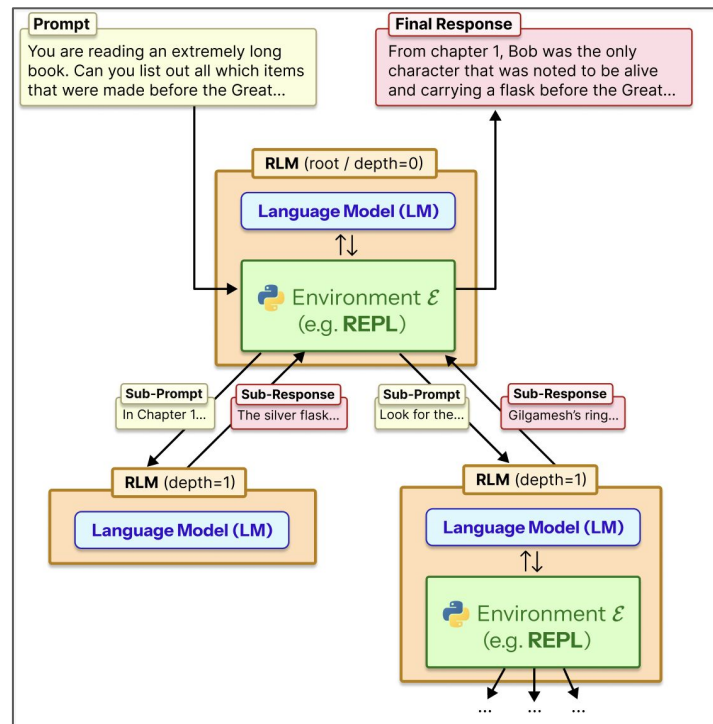
Jan 6. Cursor release dynamic context.

Recursive Language Models

Recursive Language Models (RLMs) are a general-purpose inference paradigm for

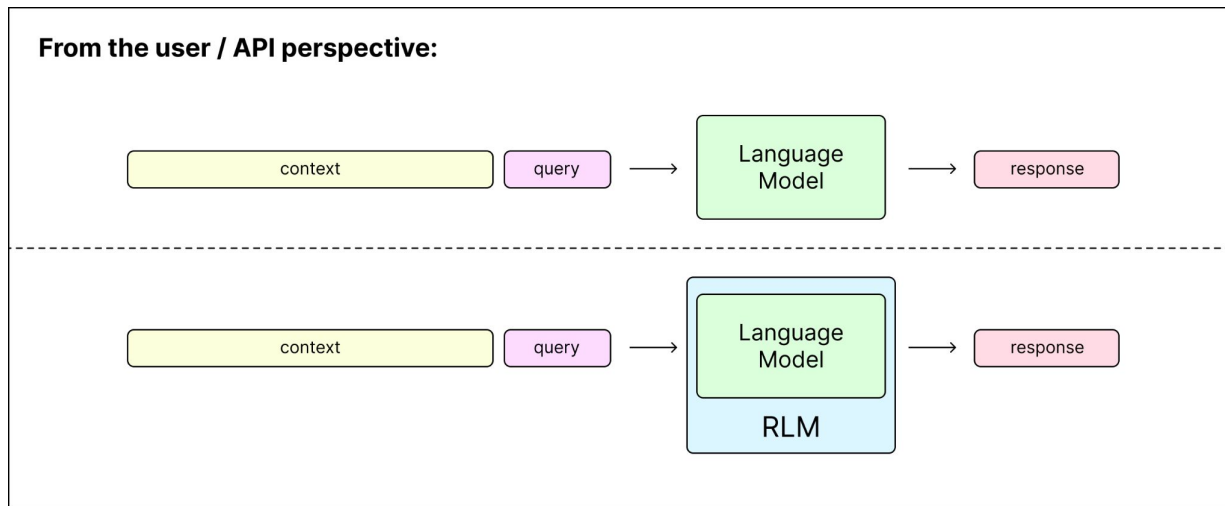
- dramatically scaling the effective input and output lengths of modern LLMs
- scaling reasoning chain patterns

The **key insight** is that long prompts should not be fed into the neural network directly but should instead be treated as part of the environment that the LLM can symbolically interact with and launch LLM calls inside of.



Recursive Language Models

- Where do we draw the line between scaffold and language model?
- $\text{RLM} \in \{(\text{probabilistic}) \text{ functions from text to text}\}$



RLMs are the new reasoning models

April 20, 2026

Reasoning models were the first clear proof that language model capability can scale with test-time compute. *Recursive language models (RLMs)* ask what the correct abstraction for spending that compute is.

The insight behind RLMs is obvious in hindsight: it is the direct marriage of two important axes of model capability — reasoning and tool use. This is more radical than it first sounds. RLMs collapse reasoning and tool use into a single inference abstraction: the model treats its own prompt as an environment it can inspect, slice, and recursively query. *Context itself becomes the object of computation.*

From Raymond Weitekamp (2026): <https://raw.works/>

Long-context view of RLMs

An argument for the correct “sub-calling” pattern in LM systems

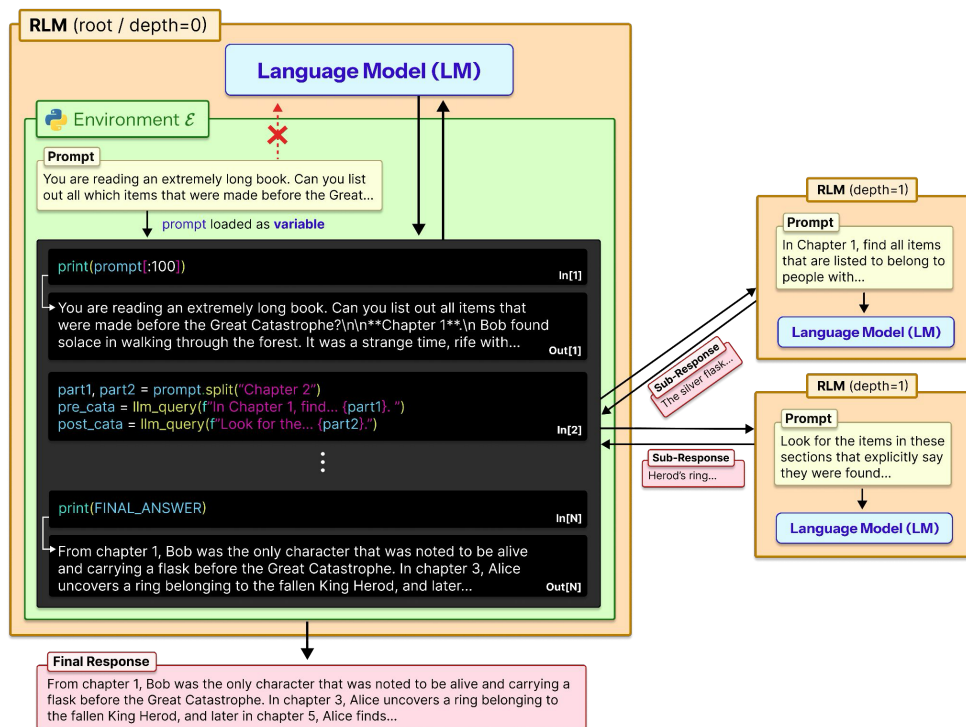
Some intuition on Recursive Language Models (RLMs)

- **RLM is a language model?** We want the **illusion** of an LM that can handle near-infinite context *without context degradation*, which works within the current limitations of LMs.
- **Can we ensure individual LM calls handle bounded inputs?** Handling long distributions of data is “long-tailed” – i.e., it rarely occurs in practice. Supporting long context natively in a NN model is very expensive.
- **Can we express sub-calling patterns with guarantees?** Certain patterns can be described better in code with sub-calls as a function (given 1M documents, launch a sub-agent only on certain documents). We want **fuzziness** of language models + **symbolic guarantees** of code.

Natively supporting “symbolic recursion”.

What are the essential components of an RLM? What has been missing?

- A **symbolic handle** to the prompt P, which offloads potentially long inputs from directly being ingested by a single language model call.
- **Persistent symbolic environment**, which allows the language model to use intermediate stateful variables.
- The ability to perform “**symbolic recursion**”, which allows the model to invoke itself using programmatic patterns to decompose and solve problems with respect to the task and/or the context.



Reasoning view of RLMs

Training a new type of “language model reasoning”

III. Reasoning view of RLMs



Arvind Narayanan
@random_walker



Why do coding agents work so well and what would it take to replicate their success in other domains? One important and under-appreciated reason is that agentic coding is a type of neurosymbolic AI.

The main weakness of LLMs is that they are statistical machines and struggle at tasks involving long chains of logic / symbol manipulation. Of course, traditional code is the opposite. The magic of agentic coding is that it fuses the two — there is a lot of code *execution* during code generation. This is a subtle point so let me spell it out.

* Most obviously, agents run the generated code itself, run tests, etc. This makes coding a verifiable domain. It is well known that in verifiable domains, inference scaling is highly effective as agents can fix their own mistakes. It also allows reinforcement learning to be highly effective.

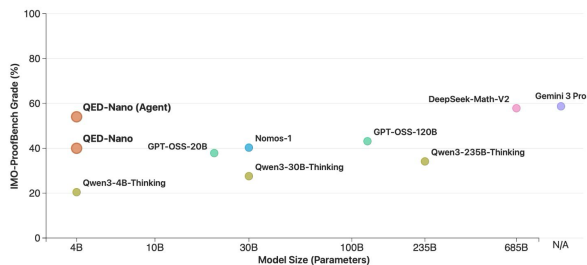
* Next, code generation often takes advantage of existing symbolic tools like compilers that have been optimized and perfected over decades. Imagine if LLMs had to directly output binary code instead. (They sometimes can, and it's a cool trick, but it's no way to do software engineering.)

* IMO the biggest neurosymbolic unlock is the shell, which allows a dramatic expansion in capabilities by using existing tools to effectively do complex editing tasks. Many of us remember the feeling of wizardry when we gained shell fluency. LLMs are able to pick up shell knowledge and best practices through pre-training because it is extensively documented on places like StackOverflow.

* Finally, more complex agentic coding tasks often involve LLMs writing code that in turn invokes LLMs. In principle you can have an arbitrary depth of recursion between statistical and symbolic systems.

Intuitively, what do reasoning models do?

QED-Nano: Teaching a Tiny Model to Prove Hard Theorems



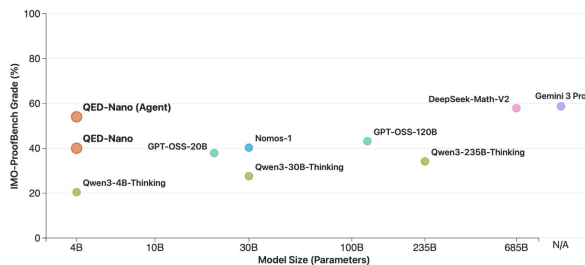
Who Needs a Trillion Parameters? Olympiad Proofs with a 4B Model 🤖

Having established that RL improves both training reward and test-time performance under the grader, the natural next step is to scale these gains further. For a small 4B model, increasing test-time computation provides a direct mechanism for extracting additional performance. A naive approach would increase the maximum response length during RL training, but this introduces substantial infrastructure costs and exacerbates variance in long-horizon optimization.

encouraging improvements in long-horizon performance. We implement this idea using the Reasoning Cache (RC) framework (Wu et al., 2026). RC decomposes reasoning into multi-step refinement cycles. At each iteration, the model generates a partial reasoning trace, summarizes its progress into a compact short textual "state representation", and conditions the next rollout on both the original problem and this summary (Figure 11). Each subsequent summarization step updates the previous summary with any information added in the current reasoning step. Then, we train the model with RL to improve its

Intuitively, what do reasoning models do?

QED-Nano: Teaching a Tiny Model to Prove Hard Theorems



Who Needs a Trillion Parameters? Olympiad Proofs with a 4B Model 🐉

Having established that RL improves both training reward and test-time performance under the grader, the natural next step is to scale these gains further. For a small 4B model, increasing test-time computation provides a direct mechanism for extracting additional performance. A naive approach would increase the maximum response length during RL training, but this introduces substantial infrastructure costs and exacerbates variance in long-horizon optimization.

encouraging improvements in long-horizon performance. We implement this idea using the Reasoning Cache (RC) framework (Wu et al., 2026). RC decomposes reasoning into multi-step refinement cycles. At each iteration, the model generates a partial reasoning trace, summarizes its progress into a compact short textual "state representation", and conditions the next rollout on both the original problem and this summary (Figure 11). Each subsequent summarization step updates the previous summary with any information added in the current reasoning step. Then, we train the model with RL to improve its

- Reasoning = conditioning model well.
Model conditioned on sufficient information will get the right answer.
- But reasoning can be wrong? Spurious rewards paper (Shao, 2026) implies incorrect reasoning traces can lead to a correct answer, but it's still safe to assume **correct reasoning traces** → **correct answer**.
- Reasoning "longer" without a contiguous chain may work! QED-Nano results and others seem to imply this may work.

Can reasoning have more guarantees + scale infinitely?

2001 AIME I Problems/Problem 14

Problem

A mail carrier delivers mail to the nineteen houses on the east side of Elm Street. The carrier notices that no two adjacent houses ever get mail on the same day, but that there are never more than two houses in a row that get no mail on the same day. How many different patterns of mail delivery are possible?



The Logic of the "o" and "|"

Let $|$ be a house with mail and o be a house without. To satisfy the "no two adjacent $|$ " rule, every $|$ (except possibly the last one) must be followed by at least one o . To satisfy the "no more than two o " rule, we can have at most one "extra" o between any two $|$, and at most two o at the very ends of the street.

Case Breakdown by Number of Mail Deliveries (k)

Case 1: $k = 10$ houses get mail

We have 10 $|$ and 9 o . To avoid $||$, the 9 o must be placed in the 9 gaps between the $|$.

Pattern: $|o|o|o|o|o|o|o|o|o|$

There is only 1 way to arrange this.

Case 2: $k = 9$ houses get mail

We have 9 $|$ and 10 o . To separate the $|$, 8 o are used as fixed separators. We have 2 "extra" o to distribute into 10 possible slots (the 8 gaps between $|$ plus the 2 ends).

- *Restriction:* Gaps between $|$ can hold at most 1 extra o (to avoid ooo). The 2 ends can hold up to 2 o total.
- Ways: $\binom{10}{2} = 45$.
- However, if we put 2 o on one end, we are effectively choosing 1 slot out of the 2 ends, then the remaining o must be placed in one of the other 9 slots: $2 \times \binom{9}{1} = 18$.

```
1 # Defining the number of houses receiving mail
2 k_values = [6, 7, 8, 9, 10]
3
4 # Generating targeted prompts for each case
5 prompts = [
6     f"In a sequence of 19 houses, exactly {k} houses receive mail (1) and {19-k} do not (0). "
7     f"Rule 1: No two 1s are adjacent. Rule 2: No three 0s are adjacent. "
8     f"How many such sequences exist for k={k}? Use stars and bars to compute this."
9     f"Provide ONLY the final integer count."
10    for k in k_values
11 ]
12
13 # Batch process the counts
14 counts_raw = llm_query_batched(prompts)
15 counts = [int(val.strip()) for val in counts_raw]
16
17 # Mapping results for clarity
18 k_results = dict(zip(k_values, counts))
19 print(f"Results per case: {k_results}")
20
21 # Calculate the final sum
22 total_patterns = sum(counts)
23 print(f"Total patterns found: {total_patterns}")
24
25 FINAL_VAR(total_patterns)
```

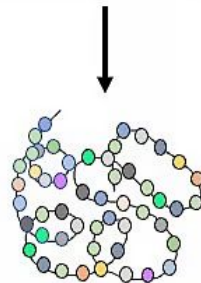
Are “LM agents” analogous to an RLM reasoning chain?

```
mini-swe-agent / src / minisweagent / agents / default.py  ↑ Top
Code Blame 155 lines (137 loc) · 6.21 KB
33 class DefaultAgent:
99     def step(self) -> list[dict]:
100         """Query the LM, execute actions."""
101         return self.execute_actions(self.query())
102
103     def query(self) -> dict:
104         """Query the model and return model messages. Override to add hooks."""
105         if 0 < self.config.step_limit <= self.n_calls or 0 < self.config.cost_limit <= self.n_calls:
106             raise LimitsExceeded()
107         {
108             "role": "exit",
109             "content": "LimitsExceeded",
110             "extra": {"exit_status": "LimitsExceeded", "submission": ""},
111         }
112     )
113     self.n_calls += 1
114     message = self.model.query(self.messages)
115     self.cost += message.get("extra", {}).get("cost", 0.0)
116     self.add_messages(message)
117     return message
118
119     def execute_actions(self, message: dict) -> list[dict]:
120         """Execute actions in message, add observation messages, return them."""
121         outputs = [self.env.execute(action) for action in message.get("extra", {}).get("actions", [])]
122         return self.add_messages(*self.model.format_observation_messages(message, outputs))
```

e.g. mini-swe-agent is ~100 lines of code!

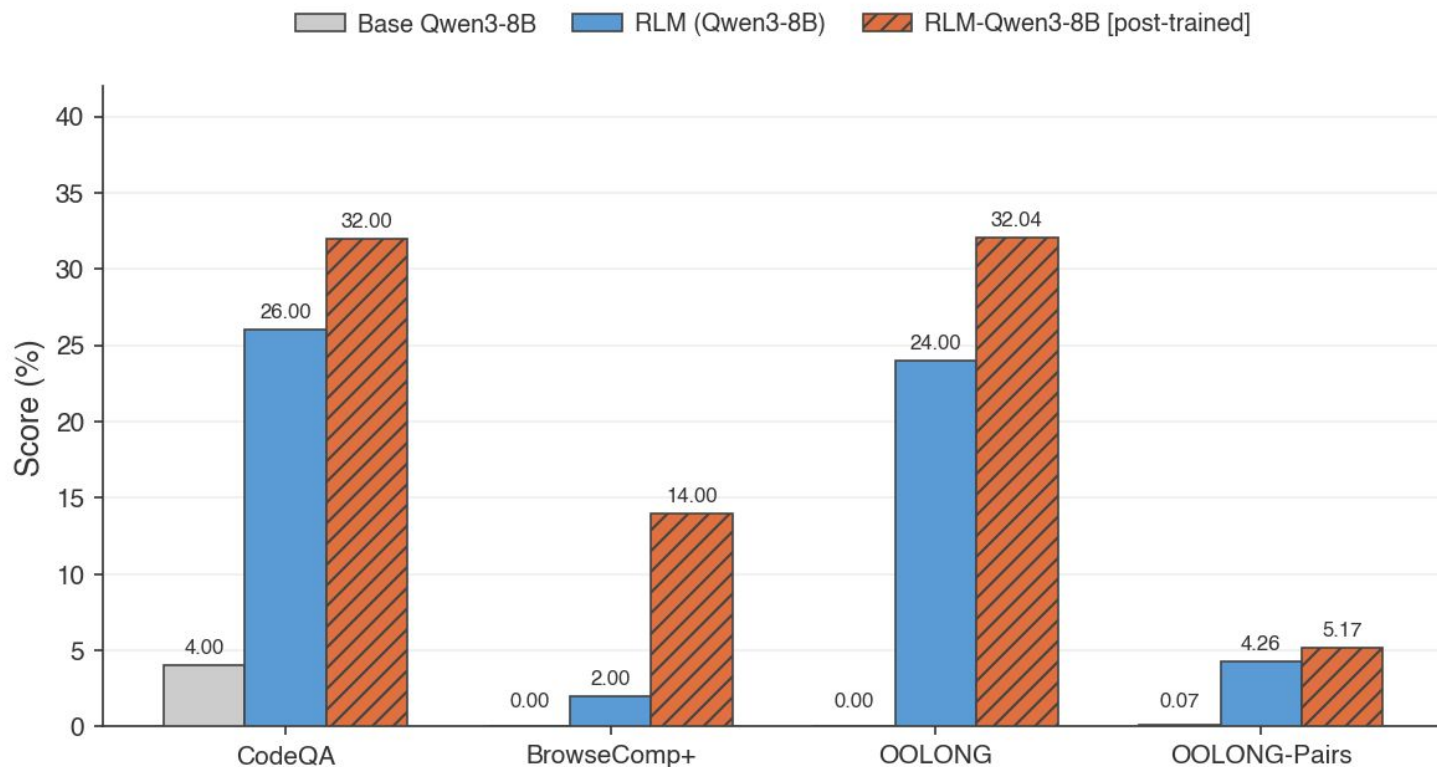
Suppose an LM is asked to produce an algorithm for *protein folding*, or some other big task.

String of amino acids (each amino acid is represented by an alphabet) forming a linear protein polymer



Folded protein with a 3-D structure: The string of amino acids of a particular sequence fold to attain a specific 3-D structure.

Small-scale Training Experiment: **RLM-Qwen3-8B-v0.1**



Lots of cool ongoing work outside our lab!



Recursive Language Models (RLMs) let agents manage 10M+ tokens by delegating tasks recursively.

This Google Cloud Community Article explains why ADK was the perfect choice for re-implementing the original RLM codebase in a more enterprise-ready format → goo.gle/4kjT12E



We believe the next breakthrough in long-horizon agents is training models to manage their own context.

Introducing our new research direction on Recursive Language Models.

We are sharing our initial experiments showing the promise of RLMs.



The dspy.RLM module is now released 🌟

Install DSPy 3.1.2 to try it. Usage is plug-and-play with your existing Signatures.

A little example of it helping @lateinteraction and I figure out some scattered backlogs:

```
rlm = dspy.RLM("discussion, request -> ideas: list[str]")

output = rlm(
    discussion=slack_dump, # past 16 months (VERY long)
    request="What are the 5 coolest unfinished ideas that Omar & Isaac keep coming back to?"
)

print(output.ideas[0])

# 1) Pre-compiled / offline-optimized adapters per LM
# (benchmark-backed, not "trust me bro") ...
```

5:00 PM · Jan 19, 2026 · 127.4K Views

29 114 471 336

Enabling RLM Inference with Shared Program State

RLM inference comes for free in programming systems using shared program state.

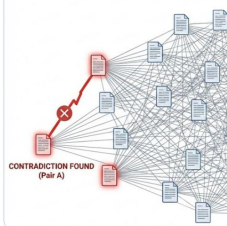
Ellie Y. Cheng (MIT CSAIL), Logan Weber (MIT CSAIL), Tian Jin (MIT CSAIL), Michael Carbin (MIT CSAIL)
Jan 20th, 2026

Two recent lines of work have revealed the benefits of marrying LLMs with programming environments, such as a Python REPL. Recursive Language Models (RLMs)[1] automates the work of managing long prompt contexts using by enabling LLMs to self-decompose context prompts. Also, our recent paper on **shared program state**[2] enables LLMs to modify the program state of a host program through a controlled interface. The common pattern of using an LLM for automated state management means programs using shared program state can easily use RLMs.

In this blog post, we show how to enable the RLM inference strategy in only a few lines of code in a program with shared program state.

OOLONG-Pairs Benchmark: Quadratic Reasoning Across Long Contexts

Task: Identify contradictory claims across all document pairs



Folks claim to set the state of the art on ARC-AGI-2 using an RLM, a deeply recursive one, to manage the long horizon.

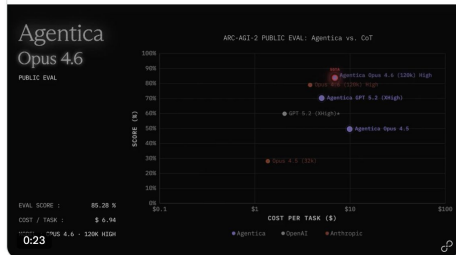
"Other agent harnesses keep everything in the model's context window. We don't. Agentic uses a stateful REPL to manage context. This is an RLM-style loop."



We set a new ARC-AGI-2 SoTA: 85.28% using an Agentic agent (~350 lines) that writes and runs code.

6:00 PM · Feb 7, 2026 · 932.8K Views

89 693



I'm hiring a student researcher to work on RL and RLM-flavored things. DM me if interested

Aman @Amank1412 · 16h
Google Student Researcher Program 2026 is now OPEN!

Work on REAL AI/ML projects with:
• Google Research
• DeepMind...



2:12 PM · Feb 23, 2026 · 29.4K Views

14 11 217 200

What we're up to now

1. **Training.** Can we train LMs to be more effective as RLMs? Can we train RLMs end-to-end to “reason” with guarantees + near-infinite length reasoning chains?
2. **Inference.** How can we make RLM inference faster and more efficient? Can we JIT compile REPL outputs for asynchrony?
3. **Domains.** Which settings are RLMs most useful? Can we use them for long-horizon tasks (e.g. agentic, evolutionary methods, etc.)